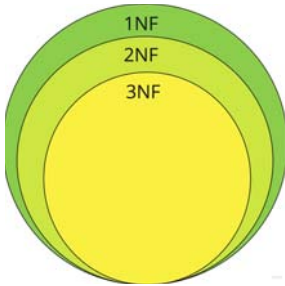


A 3.2.5 Normal Forms



A3.2.5 Explain the difference between normal forms.

This section explains normal forms (1NF, 2NF, 3NF) in relational databases. It covers atomicity, unique identification, functional/partial-key/transitive dependencies. Normalization reduces data duplication, missing data, and dependency issues to ensure data integrity.

Introduction

Normalization is a database design technique that reduces data redundancy and improves data integrity by organizing data into tables in a structured way. This process involves dividing larger tables into smaller, more manageable tables and defining relationships between them. Normal forms are a series of guidelines that help achieve this organisation

First normal form (1NF), second normal form (2NF), third normal form (3NF)

First Normal Form (1NF)

A table is in 1NF when:

- Each column is atomic. This means the values in the column cannot be broken down further into meaningful parts.
- Each cell holds only one value. No lists, no repeating groups.
- Every row must be uniquely identifiable, which usually means the table must have a Primary Key.
- Consistent data types must be used in a column.

For example:

Unnormalized Table (Not in 1NF - Repeating groups):

CustomerID	CustomerName	OrderIDs	OrderDates
1	Alice	101, 102	2024-03-01, 2024-03-15
2	Bob	103	2024-03-20

See how Customer 1 has two OrderIDs and two OrderDates in the same cell? That's a repeating group, and it's not atomic.

1NF Table:

CustomerID	CustomerName	OrderIDs	OrderDates
1	Alice	101	2024-03-01
1	Alice	102	2024-03-15
2	Bob	103	2024-03-20

Now, each row is about one order, and each cell has only one value.

Notice also that each row is still identified uniquely with a combination of CustomerID and OrderID which is known as a Composite Key.

Unnormalized Table (Not in 1NF - Not Atomic):

EmployeeID	FullName	Address
1	John Smith	123 Main St, Anytown, CA
2	Jane Doe	456 Oak Ave, Othercity, NY

See how "FullName" has first and last name, and "Address" has street, city, and state? These can be broken down further.

EmployeeID	FirstName	Surname	Street	City	Sate
1	John	Smith	123 Main St	Anytown	CA
2	Jane	Doe	456 Oak Ave	Othercity	NY

Now, "FullName" and "Address" are broken down into atomic parts.

Notice each row is uniquely identified by the EmployeeID which is a Primary Key.

Key Idea: One cell, one value, and that value can't be meaningfully broken down.

Second Normal Form (2NF)

A table is in 2NF if it is in 1NF and all non-key attributes are fully functionally dependent on the entire primary key. 2NF applies to tables with composite primary keys. If a table has a simple primary key, it is automatically in 2NF.

- 2NF only matters if your table has a composite primary key. That's a primary key made up of two or more columns.
- 2NF says: If you have a composite primary key, every non-key column must depend on all parts of that key, not just some of it.
- Think of it as: Don't let any column be lazy and only rely on part of the ID.

Key Concepts:

- **Functional Dependency:** An attribute (B) is functionally dependent on another attribute (A) if, for each value of A, there is only one possible value of B. (A → B).
 - **Example:** Knowing a "CustomerID" tells you the "CustomerName."
- **Partial-Key Dependency:** A non-key attribute is partially dependent on a primary key if it is only dependent on a part of a composite primary key.

Example:

1NF Table (but not 2NF if we assume OrderID and ProductID together are the PK)

OrderID	ProductID	ProductName	OrderDate
101	A1	Widgit	2024-03-01
102	A2	Gadget	2024-03-01
103	A1	Widgit	2024-03-15

In this case, **ProductName** is only dependent on **ProductID**, not **OrderID**. This is a partial dependency.

2NF Tables:

OrderID	ProductID	OrderDate
101	A1	2024-03-01
102	A2	2024-03-01

OrderID	ProductID	OrderDate
103	A1	2024-03-15

ProductID	ProductName
A1	Widget
A2	Gadget

Key Idea: If you have a composite key, make sure every column depends on *all* parts of it.

Third Normal Form (3NF)

A table is in 3NF if it is in 2NF and no non-key attribute is transitively dependent on the primary key.

Key Concept:

Transitive Dependency: If $A \rightarrow B$, and $B \rightarrow C$, then C is *transitively* dependent on A .

- A = Primary Key
- B = Non-Key Column
- C = Another Non-Key Column

Example:

2NF Table (but not 3NF):

CustomerID	CustomerName	CustomerCity	CityZipCode
1	Alice Smith	Anytown	12345
2	Bob Johnson	Othercity	67890

Dependencies:

- **CustomerName** depends on **CustomerID** (knowing the customer ID tells you the name).
- **CustomerCity** depends on **CustomerID** (knowing the customer ID tells you the city).
- **Crucially:** **CityZipCode** depends on **CustomerCity** (knowing the city tells you the zip code).

See the problem? **CityZipCode** is "lazy" again! It doesn't directly depend on the primary key (**CustomerID**). It goes through **CustomerCity** to get there. This is a transitive dependency.

So we need to split the table again.

3NF Tables:

CustomerID	CustomerName	CustomerCity
1	Alice Smith	Anytown
2	Bob Johnson	Othercity

CityName	CityZipCode
Anytown	12345
Othercity	67890

- Now, **Customers** only has columns that depend directly on **CustomerID**.
- And **Cities** stores city/zip code info once, avoiding redundancy.

Key Idea: No non-key column should depend on another non-key column. All non-key columns must depend directly on the primary key.

Here's the key difference between moving to 2NF and moving to 3NF:

- **2NF is about partial dependencies.**
 - This only applies when you have a composite primary key.
 - It's about a non-key column depending on *part* of the primary key, not all of it.
 - You fix it by splitting the table so that every non-key column *fully* depends on the entire composite primary key.
- **3NF is about transitive dependencies.**
 - This can happen with *any* primary key (simple or composite), but it's easier to see with a simple one.
 - It's about a non-key column depending on *another* non-key column, not directly on the primary key.
 - You fix it by splitting the table so that every non-key column depends *only* on the primary key.

The Terms Atomicity, Unique Identification, Functional Dependencies, Partial-Key Dependencies, Non-Key/Transitive Dependencies

Atomicity:

The property of data values being indivisible. A column is atomic if it cannot be further subdivided into meaningful parts. (Relates to 1NF)

Unique Identification:

The ability to distinguish each row in a table from all other rows, achieved through the use of primary keys.

Functional Dependency:

A relationship between attributes where the value of one attribute (the determinant) determines the value of another attribute (the dependent). ($A \rightarrow B$)

Partial-Key Dependency:

A dependency where a non-key attribute is dependent on only a part of a composite primary key. (Violates 2NF)

Non-Key/Transitive Dependency:

A dependency where a non-key attribute is dependent on another non-key attribute, which is dependent on the primary key. (Violates 3NF)

Normalization Issues

Normalization addresses several potential issues in database design:

- **Data Duplication:**
 - Storing the same data multiple times, leading to wasted space and inconsistencies.
- **Missing Data:**
 - Normalization can help in structuring the database to minimize the possibility of missing crucial information.
- **Dependency Concerns:**

- **Data Dependencies:** General term for attribute relationships.
- **Composite Key Dependencies:** Issues related to keys with multiple attributes.
- **Transitive Dependencies:** As defined above, causing update anomalies.
- **Multi-Valued Dependencies:** When an attribute depends on another attribute in a way that creates multiple values. (Beyond the scope of 3NF)

Key Questions

Explain the purpose of normalization in database design.

The purpose of normalization is to reduce data redundancy and improve data integrity.

It achieves this by organizing data into tables in a structured way, minimizing the duplication of data and the potential for inconsistencies. Normalization also helps to eliminate update, insert, and delete anomalies.

What is the difference between 1NF and 2NF?

A table is in 1NF if all its columns contain atomic values (no repeating groups).

A table is in 2NF if it meets the requirements of 1NF and, in addition, all non-key attributes are fully functionally dependent on the entire primary key. This distinction is only relevant for tables with composite primary keys; if a table has a simple primary key, it automatically satisfies 2NF. 2NF eliminates partial dependencies.

Describe a scenario where a transitive dependency might cause an update anomaly.

Scenario: A table with **CustomerID**, **CustomerCity**, and **CityZipCode**. **CityZipCode** is transitively dependent on **CustomerID** (**CustomerID** → **CustomerCity** → **CityZipCode**).

Update Anomaly: If you need to update the **CityZipCode** for a city, you might have to update it in multiple rows if there are multiple customers from that city. If you fail to update all rows, you'll have inconsistent data (some customers from the same city with different zip codes).

Why is atomicity important in the context of 1NF?

Atomicity is important in 1NF because it simplifies data management and manipulation.

When each column contains only a single value, it's easier to search, sort, and filter data. It also prevents the need for complex parsing or extraction of data from within a single field. For example,

if a "Name" field contains both first and last names, you would need extra steps to sort by last name, which is avoided with atomic columns.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**